

STOCKPILOT LEARNING SERIES

# StockPilot Backend Handbook

A beginner-friendly guide to the API, quantitative rules, AI agents, database, backtesting, security, testing, and deployment.

Use this document to

- Learn the codebase from input to response
- Understand which parts are AI and which are deterministic
- Prepare clear, honest interview answers

API: <https://finance-stock-analyzer.onrender.com/docs>

# How to use this handbook

Read once in order for understanding. Before an interview, review the diagrams, the scoring example, the backtesting integrity checks, and the short-answer section.

## Goal

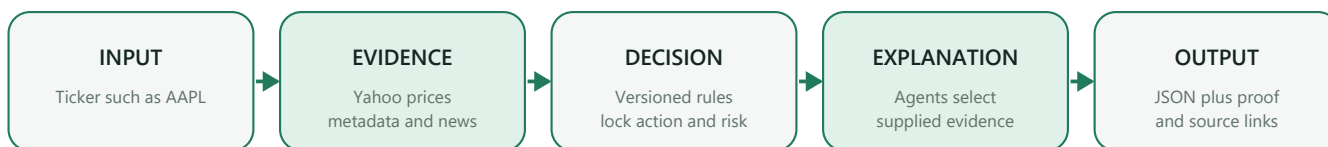
After reading this guide, you should be able to run the project, explain every major design decision, diagnose common failures, and answer interview questions without memorizing buzzwords.

## Learning path

- 01 Project purpose and system boundaries
- 02 Architecture and file map
- 03 Analysis request from input to response
- 04 Market data and indicators
- 05 Rules-based decision engine
- 06 CrewAI agents and hallucination controls
- 07 Authentication and API design
- 08 Database and personal data
- 09 Backtesting methodology
- 10 Jobs, caching, limits, and errors
- 11 Docker and production deployment
- 12 Local setup and testing
- 13 Troubleshooting
- 14 Interview questions and model answers
- 15 Limitations and responsible presentation

# 1. What the backend does

The backend is the trusted calculation and data layer of StockPilot. It accepts authenticated requests, collects market evidence, calculates reproducible indicators, optionally asks constrained agents to review that evidence, stores user-owned results, and returns JSON to the frontend.



The signal is calculated before the AI explanation is created.

The backend is not a trading bot. It does not place orders, manage brokerage accounts, predict guaranteed returns, or provide personalized financial advice.

## Core design

Numbers come from code and market data. AI is used as a constrained reviewer, not as the source of prices, indicators, or the final action.

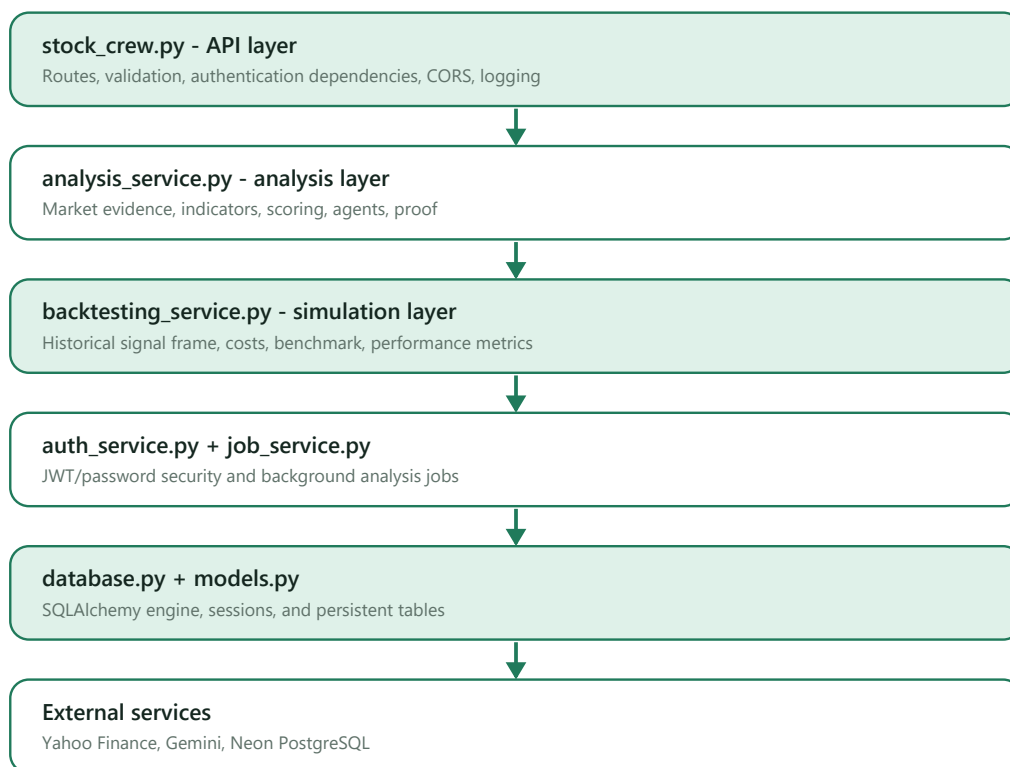
## Main responsibilities

- Validate tickers and reject malformed input.
- Authenticate users and protect personal endpoints.
- Calculate indicators and a versioned technical score.
- Run background AI analysis without blocking the browser.
- Persist users, watchlists, analyses, and backtests.
- Expose calculation proof and source metadata.

## 2. Architecture and file map

Each module has one main responsibility. This separation makes the code easier to test, explain, and replace.

### Logical backend layers



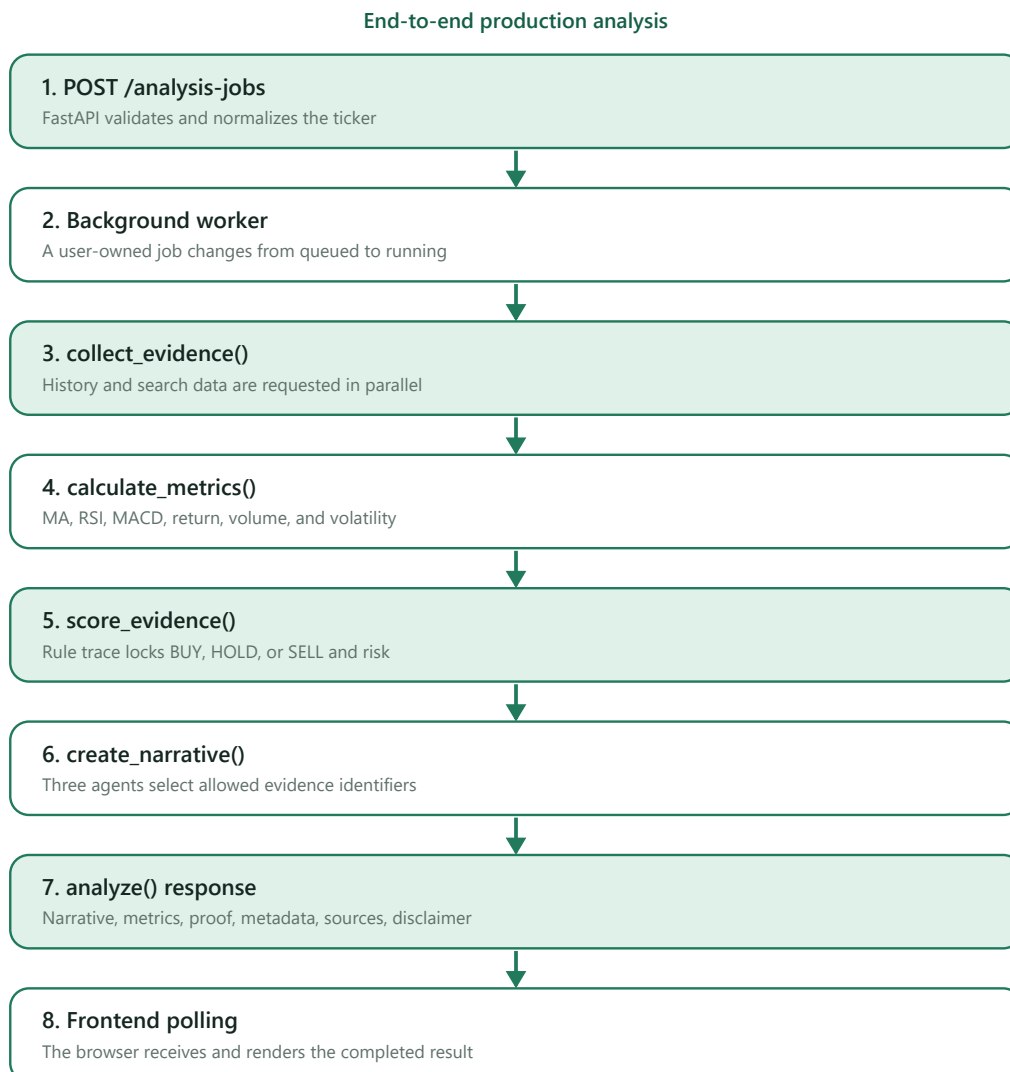
| File                   | What to remember                                                          |
|------------------------|---------------------------------------------------------------------------|
| stock_crew.py          | FastAPI application and HTTP contract.                                    |
| analysis_service.py    | Current production analysis path and proof generation.                    |
| strategy_config.py     | Shared, configurable thresholds and rule version.                         |
| stock_agents.py        | CrewAI agent definitions. Production imports Research, Risk, and Advisor. |
| backtesting_service.py | No LLM; deterministic historical simulation.                              |
| models.py              | User-owned PostgreSQL entities.                                           |
| runtime_config.py      | Production safety checks.                                                 |
| tests/                 | Isolated automated verification.                                          |

### Repository detail

Older Data Collector, Technical Analyst, and News Analyst definitions remain in `stock_agents.py` and `stock_tasks.py`. The current production `analyze()` path does not use those tasks; it uses deterministic collectors plus the Research, Risk, and Advisor agents.

### 3. From ticker input to final response

This is the most important flow to explain in an interview.



Price history and search run concurrently to reduce waiting time. Evidence is cached for five minutes by ticker. If the AI provider fails, the application returns a deterministic fallback narrative instead of losing the calculated signal.

## 4. Market data and technical indicators

Yahoo Finance is the evidence provider. yfinance is a Python client that retrieves adjusted daily price history, volume, company metadata, and recent news.

| Term           | Simple meaning                                | How StockPilot uses it                         |
|----------------|-----------------------------------------------|------------------------------------------------|
| Adjusted close | Price corrected for splits and distributions. | Consistent return and indicator calculations.  |
| MA20           | Average of the latest 20 closes.              | Shorter-term trend.                            |
| MA50           | Average of the latest 50 closes.              | Medium-term trend.                             |
| RSI(14)        | Momentum oscillator from 0 to 100.            | 70+ is overbought; 30- is oversold by default. |
| MACD           | 12-day EMA minus 26-day EMA.                  | Compared with its 9-day signal line.           |
| Volatility     | Variation of daily returns, annualized.       | Determines LOW, MEDIUM, or HIGH risk.          |
| 3-month change | Change from roughly 63 sessions earlier.      | Evidence candidate, not a separate score rule. |

### Key formulas

```
MA20 = mean(latest 20 adjusted closes)
```

```
MACD = EMA(12) - EMA(26)
```

```
Annualized volatility = standard deviation(daily returns) x sqrt(252) x 100
```

#### Why adjusted prices?

They reduce artificial jumps caused by corporate actions, making historical comparisons and backtests more meaningful.

## 5. The rules-based decision engine

The action is reproducible: the same metrics and thresholds produce the same score. The active rule version is technical-rules-v2.

| Rule  | Condition           | Points |
|-------|---------------------|--------|
| Trend | Price > MA20 > MA50 | +2     |
| Trend | Price < MA20 < MA50 | -2     |
| RSI   | RSI >= 70           | -1     |
| RSI   | RSI <= 30           | +1     |
| MACD  | MACD > signal line  | +1     |
| MACD  | Otherwise           | -1     |

### Final action



### Worked example

Suppose price > MA20 > MA50 (+2), RSI is 74 (-1), and MACD is above its signal (+1). Total score = +2, therefore action = BUY. The response still warns that RSI is overbought.

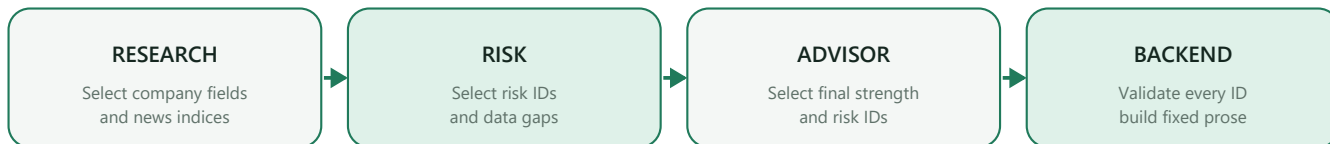
Risk is separate from action: volatility below 20% is LOW, 20% to below 40% is MEDIUM, and 40% or higher is HIGH by default. A BUY signal can still have HIGH risk.

#### Interview phrase

The model separates direction from uncertainty: the technical score chooses the action, while historical volatility chooses the risk category.

## 6. CrewAI agents: what they really do

An agent is more than an LLM name. It is an LLM configured with a role, goal, task, context, constraints, and optional tools. CrewAI coordinates multiple agents and passes task outputs between them.



Research and Risk tasks are asynchronous; Advisor considers both outputs.

| Agent                     | Allowed work                                                    | Not allowed                                  |
|---------------------------|-----------------------------------------------------------------|----------------------------------------------|
| Evidence Research Analyst | Choose supplied company fields, news indices, and caveat codes. | Invent facts, prices, or recommendations.    |
| Independent Risk Auditor  | Choose supplied risk IDs and data-gap codes.                    | Change action/risk or create new claims.     |
| Senior Investment Advisor | Choose final supplied strength and risk IDs.                    | Write factual prose or overwrite the signal. |

The backend parses agent JSON, rejects unknown identifiers, limits selections, and constructs the final summary from deterministic text candidates. If parsing or Gemini fails, `_fallback_narrative()` uses verified metrics.

### Correct answer

Yahoo Finance collects the data. Python calculates the numbers. CrewAI agents review which supplied evidence is most relevant. They do not create the market data.

## 7. How results are verified

Verification is built into the response rather than hidden inside the application.

- Ticker validation prevents arbitrary or malformed input.
- Numeric values are calculated from adjusted price history and normalized to finite values.
- The proof object records each observed value, condition, and score contribution.
- Decision thresholds and the rule version are returned to the client.
- News must contain a title, publisher, and valid HTTP/HTTPS URL; duplicates are removed.
- Agent identifiers are checked against allowlists before use.
- The metadata records source, source URL, exact date window, trading-day count, timestamp, and cache status.
- Tests replace external providers with controlled data so calculations are repeatable.

### Evidence-quality score

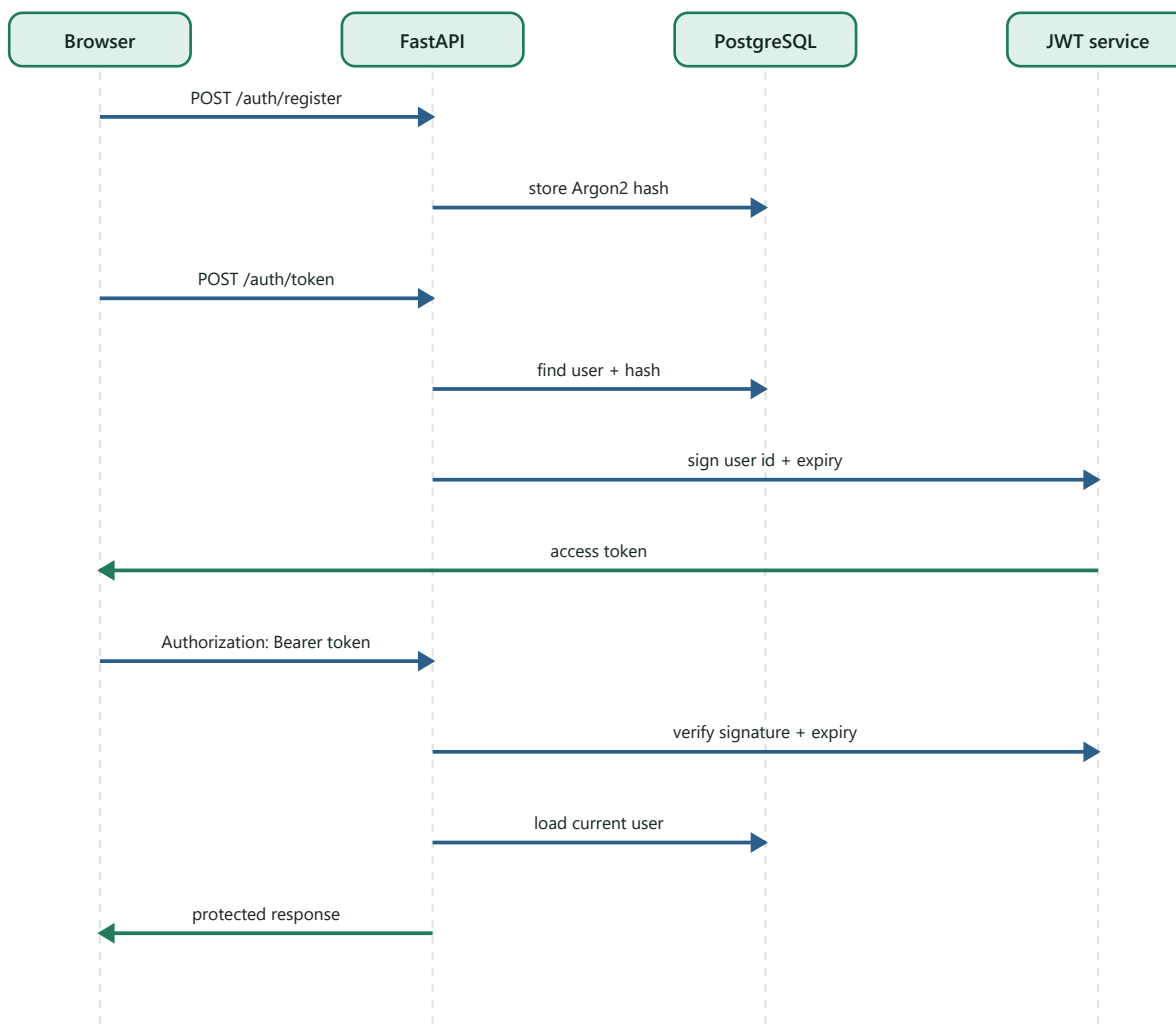
| Component              | Maximum |
|------------------------|---------|
| Base                   | 30      |
| Core market metrics    | 35      |
| Company metadata       | 10      |
| Attributable news      | 10      |
| Completed agent review | 15      |

#### Do not misinterpret

Evidence quality measures completeness and review coverage. It is not the chance that the stock will rise and not a model accuracy percentage.

## 8. Authentication and API design

FastAPI provides typed request validation and Swagger documentation. Protected routes use OAuth2 bearer tokens containing a signed JWT.

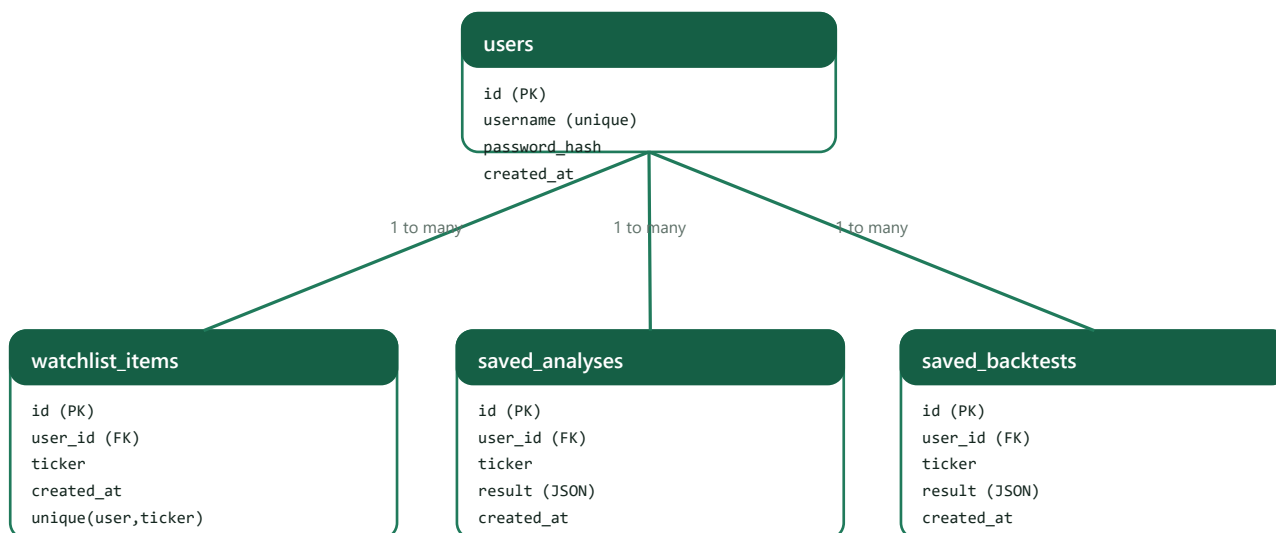


Passwords are never stored directly. `pwdlib` uses a recommended Argon2 password hash. JWTs use HS256 and expire after 60 minutes by default. The server reads the user ID from the token, then loads the user from the database.

| Status      | Meaning in StockPilot                        |
|-------------|----------------------------------------------|
| 200/201     | Successful read or creation.                 |
| 204         | Successful deletion with no response body.   |
| 401         | Missing, invalid, or expired token.          |
| 409         | Duplicate username or watchlist ticker.      |
| 422         | Validation or insufficient market history.   |
| 429         | Rate limit reached.                          |
| 502/503/504 | Provider, configuration, or timeout failure. |

## 9. Database and personal data

SQLAlchemy maps Python classes to tables. SQLite is the local default; production uses Neon PostgreSQL through DATABASE\_URL.



Each saved record carries a `user_id` foreign key. Every portfolio query filters by the authenticated user, preventing one user from reading another user's data. Deleting a user cascades to related records in the ORM model.

### Why PostgreSQL in production?

Render's container filesystem is temporary. Neon stores accounts, watchlists, and saved results independently, so data survives backend restarts and deployments.

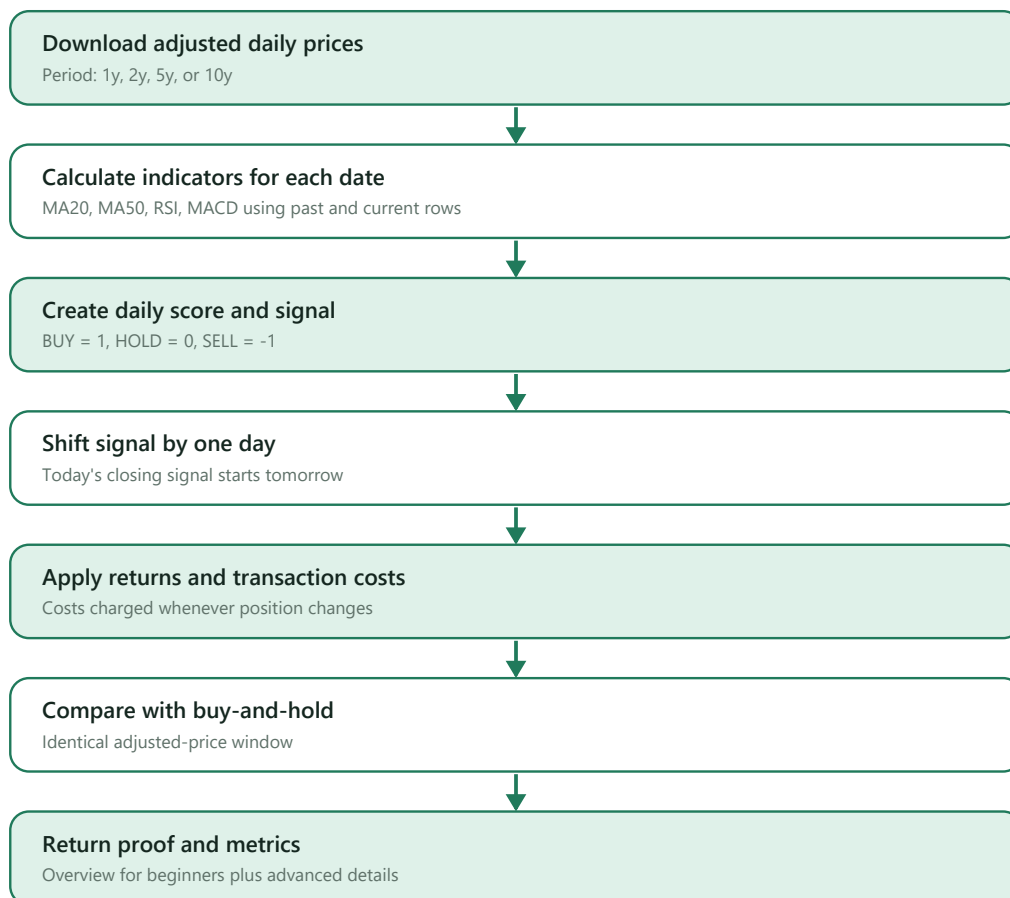
### Storage limit

The application itself does not impose a fixed record count. The practical limit is the Neon plan's storage and compute allowance. JSON results are convenient for a demo because they preserve the exact response shown at save time.

## 10. Backtesting engine

Backtesting asks: if the same rules had been applied to historical prices, how would the simulated strategy have behaved compared with simply buying and holding? No LLM participates.

Look-ahead bias prevention is the key integrity control.



| Metric                | Plain-English meaning                                                |
|-----------------------|----------------------------------------------------------------------|
| Total return          | Overall percentage gain or loss.                                     |
| Annualized return     | Compound return expressed per year.                                  |
| Annualized volatility | Yearly variability of daily returns.                                 |
| Sharpe ratio          | Return per unit of volatility; assumes 0% risk-free rate here.       |
| Maximum drawdown      | Worst peak-to-trough decline.                                        |
| Exposure              | Percentage of test days with a non-zero position.                    |
| Signal hit rate       | Share of eligible BUY/SELL signals correct after the chosen horizon. |

# 11. Jobs, caching, limits, and failures

These mechanisms make the demo faster and safer without adding Redis or a distributed queue.

## Asynchronous jobs

POST /analysis-jobs returns a job ID immediately. A background thread runs the analysis. GET /analysis-jobs/{id} reports queued, running, completed, or failed. Jobs are held in memory and belong to the creating user.

## Caching

Market evidence is cached by ticker for five minutes. Repeated requests can reuse data, reduce provider calls, and complete faster. The response reports whether cached evidence was used.

## Rate limiting

In-memory counters limit registration, login, analysis, and backtesting by user or IP. This protects the public demo and Gemini free quota. Counters reset when the single server instance restarts.

## Failure behavior

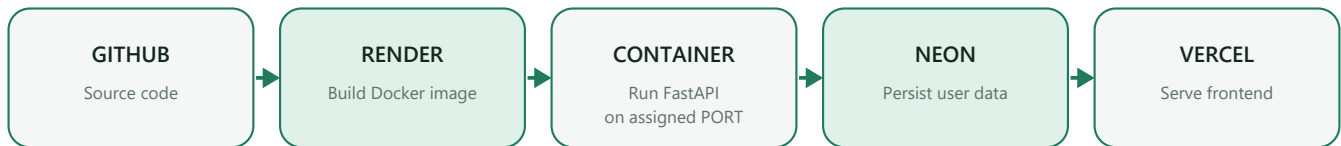
| Failure                  | Behavior                                          |
|--------------------------|---------------------------------------------------|
| Gemini unavailable       | Return deterministic fallback narrative.          |
| Yahoo search fails       | Try ticker-specific news; omit news if both fail. |
| Price history times out  | Return a gateway-timeout error.                   |
| Database unhealthy       | Health check reports unhealthy.                   |
| Unsafe production config | Application refuses to start.                     |

### Tradeoff

In-memory jobs and limits are appropriate for a single-instance demo. A multi-instance production system would move them to a durable queue and shared store such as Redis.

## 12. Docker and deployment

Docker packages the code, Python runtime, dependencies, and startup instruction into a reproducible image. It is not the permanent database.



| Platform      | Responsibility                                               |
|---------------|--------------------------------------------------------------|
| Vercel        | Builds and serves the Next.js frontend.                      |
| Render        | Builds the Dockerfile and runs the FastAPI container.        |
| Neon          | Hosts PostgreSQL outside the temporary container filesystem. |
| Google Gemini | Provides LLM inference for constrained agent review.         |
| Yahoo Finance | Provides market evidence.                                    |

### Required production configuration

```
APP_ENV=production
GEMINI_API_KEY=<private>
JWT_SECRET=<at least 32 unique characters>
DATABASE_URL=<private Neon connection string>
CORS_ORIGINS=https://stockpilot-analyzer.vercel.app
```

Production startup rejects a missing Gemini key, weak JWT secret, wildcard CORS, or an invalid application environment. Render free instances may sleep, so the first request can take longer.

## 13. Local setup and verification

Run the backend independently before starting the frontend.

1. Clone the backend repository and open it in VS Code.
2. Create a virtual environment: `python -m venv .venv`
3. Activate it: `.\venv\Scripts\Activate.ps1`
4. Install dependencies: `pip install -r requirements.txt`
5. Copy `.env.example` to `.env` and add your own Gemini key and JWT secret.
6. Start the API: `uvicorn stock_crew:app --reload`
7. Open `http://127.0.0.1:8000/docs` and test `/health`.

### Run the complete test suite

```
python -m unittest discover -s tests -v
```

The suite uses an isolated in-memory SQLite database and controlled provider results. It covers analysis, authentication, authorization, portfolio persistence, database URL normalization, backtesting integrity, asynchronous jobs, and rate limits.

#### Current verification

27 tests pass in the documented project state. A test suite proves specified behavior for covered cases; it does not prove that every future market response or deployment condition will succeed.

# 14. Troubleshooting map

Diagnose from the boundary closest to the error: browser, API, provider, or database.

| Symptom                          | Likely cause                                  | Check                                                 |
|----------------------------------|-----------------------------------------------|-------------------------------------------------------|
| Failed to fetch                  | CORS or wrong backend URL                     | Vercel NEXT_PUBLIC_API_URL and Render CORS_ORIGINS.   |
| 401                              | Bad/expired token                             | Sign in again; verify JWT_SECRET is stable.           |
| 429                              | Quota or app rate limit                       | Provider billing/quota and configured request limits. |
| News empty                       | Yahoo endpoint returned no attributable items | Render logs and new analysis after cache/deploy.      |
| Watchlist empty after deployment | New production database/account               | Add a ticker; local SQLite data is separate.          |
| Slow first request               | Render cold start plus providers              | Wait for health, then retry.                          |
| Database unhealthy               | Bad Neon URL/network                          | DATABASE_URL, SSL options, Neon status.               |
| Uvicorn not recognized           | Virtual environment inactive                  | Activate .venv or use python -m uvicorn.              |

# 15. Backend interview questions

Answer with the problem, the design choice, and the tradeoff. The model answers below are intentionally short enough to say aloud.

## Q1. Explain StockPilot in one minute.

StockPilot is an evidence-grounded stock research application. FastAPI collects Yahoo Finance data, calculates versioned technical indicators and a deterministic signal, then uses constrained CrewAI agents with Gemini to select relevant evidence. It returns the result with formulas, rule traces, sources, and a disclaimer. PostgreSQL stores users and saved research, while a separate deterministic engine backtests the same rules.

## Q2. Is the recommendation generated by AI?

No. BUY, HOLD, or SELL and the risk category are calculated before the LLM runs. Agents select from allowlisted evidence identifiers. This keeps the important numbers reproducible while still providing an intelligent review layer.

## Q3. Why use CrewAI instead of one prompt?

CrewAI gives separate responsibilities to Research, Risk, and Advisor roles and lets the final task consider specialist outputs. The tradeoff is extra latency and provider calls, so tasks are constrained and failures have a deterministic fallback.

## Q4. How do you reduce hallucination?

Agents return identifiers rather than free factual prose. The server validates identifiers against known candidates, constructs the final text from verified values, requires attributable news URLs, exposes proof, and falls back to deterministic text when agent output is invalid.

## Q5. Why FastAPI?

FastAPI provides type-based validation, dependency injection for authentication and database sessions, async-capable HTTP handling, automatic OpenAPI documentation, and clear status-code contracts.

## Q6. Why JWT?

JWT gives a signed, expiring identity token that the browser can send with protected requests. It avoids a server-side session table for this demo. The tradeoff is that token revocation is harder, so expiration is short and HTTPS is required in production.

# Backend interview questions - continued

## Q7. Why hash passwords with Argon2?

Passwords should never be stored reversibly. Argon2 is deliberately expensive and memory-hard, which makes brute-force attacks more costly. Login verifies the submitted password against the stored hash.

## Q8. Why PostgreSQL instead of SQLite in production?

SQLite is excellent for zero-configuration local work, but Render's filesystem is temporary and concurrent server access is better handled by a database service. Neon PostgreSQL persists independently across deployments.

## Q9. How do you prevent look-ahead bias?

Indicators use data available through each closing date, and the signal is shifted by one trading day before it earns returns. Therefore today's close cannot earn today's already-completed return.

## Q10. What does Sharpe ratio mean here?

It is average daily strategy return divided by daily return standard deviation, annualized by  $\sqrt{252}$ . This implementation assumes a 0% risk-free rate, which is disclosed in methodology.

## Q11. What does Docker solve?

Docker makes the Python runtime, dependencies, code, and start command reproducible across machines. The container runs the API; persistent data stays in Neon because container storage is temporary.

## Q12. What happens when Gemini fails?

The quantitative signal and proof are already complete. The code catches the agent failure and creates a fallback summary from verified metrics, marking agent status as fallback.

## Q13. Why are jobs in memory?

The project targets a single-instance demo, so an in-memory job store is simple and sufficient. Jobs reset on restart. At scale I would use a durable queue such as Celery or RQ with Redis and store job state persistently.

# Backend interview questions - deeper follow-ups

## Q14. How is evidence quality calculated?

It combines a base score, completeness of six core market metrics, company metadata, attributable news count, and whether agent review completed. It measures evidence coverage, not prediction accuracy.

## Q15. Why share `strategy_config.py`?

Live analysis and backtesting must use the same thresholds. A shared configuration prevents silent drift and returns a rule version so results can be audited later.

## Q16. How does authorization differ from authentication?

Authentication proves who the user is using a JWT. Authorization ensures database queries and job lookups are filtered by that user's ID, so authenticated users still cannot access each other's records.

## Q17. What are the biggest limitations?

The signal is technical rather than a complete valuation model, Yahoo Finance is an unofficial data dependency, free providers and hosts can be slow, in-memory jobs do not scale horizontally, and a backtest cannot prove future performance.

## Q18. How would you scale it?

Use a shared cache and distributed rate limiter, move analysis to a durable job queue, add database migrations and connection pooling, introduce provider retries and observability, run multiple API instances, and add continuous integration.

## Q19. How do tests avoid cost and instability?

They mock Yahoo and analysis workers and use in-memory SQLite. That keeps tests deterministic, fast, and independent from Gemini quota and live network behavior.

## Q20. What are you most proud of?

A strong answer is the trust boundary: AI makes the explanation smarter, but deterministic code owns the financial numbers and exposes proof. This is more defensible than presenting free-form LLM output as analysis.

# 16. Limitations and responsible presentation

A strong portfolio project is credible because it explains what it cannot do.

- Yahoo Finance availability and response formats can change.
- Technical indicators summarize historical prices; they do not know future events.
- The current model does not perform a complete cash-flow valuation or portfolio optimization.
- Backtests can be affected by parameter choices, survivorship considerations, taxes, spreads, and market impact.
- Free hosting may sleep and free AI quotas may change.
- The system is educational and should not be marketed as professional financial advice.

## Good next engineering steps

- Add GitHub Actions for automated backend and frontend validation.
- Add Alembic migrations for explicit database schema evolution.
- Add structured metrics, uptime monitoring, and provider latency dashboards.
- Version and save strategy parameters with every analysis and backtest.
- Replace in-memory jobs only if the project needs multiple backend instances.

### Final revision sentence

StockPilot uses deterministic quantitative rules for decisions, constrained AI agents for evidence review, transparent proof for trust, and separate cloud services for the frontend, API, and persistent data.

Live application: <https://stockpilot-analyzer.vercel.app>

Backend repository: <https://github.com/mayank2OP/finance-stock-analyzer>

API documentation: <https://finance-stock-analyzer.onrender.com/docs>